SDEV2150

Lesson 13: Lifting State and Shared State

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

Lesson Objectives

In this lesson, students will:

- Explain the concept of lifting state and when it should be applied
- Pass state and event handlers between parent and child components
- Identify problems caused by duplicated state
- Describe the challenges of prop drilling

Agenda

- Recap: Local state and events
- The problem of duplicated state
- Identifying shared state
- Lifting state to a common ancestor
- Data flow patterns in React
- Prop drilling discussion
- Exit ticket

CONNECTING

WE ARE ESSENTIAL TO ALBERTA

NAIT

Review from last lesson

- Components can manage local state
- State changes trigger re-renders
- Events update state

But what happens when multiple components need the same data?

THE CORE PROBLEM

WE ARE ESSENTIAL TO ALBERTA | NAIT

Duplicated state leads to inconsistency

If two components store the same piece of data:

- They can fall out of sync
- UI becomes unpredictable
- Debugging becomes harder

State should exist in one place.

IDENTIFYING SHARED STATE

What's an example of a user interaction that requires state to update the UI?

Ask two questions:

1. Who needs this data?
2. Where is their nearest common ancestor?

The answer determines where state should live.

LIFTING STATE UP

What it means

Move shared state from child components to their nearest common parent.

The parent becomes the single source of truth.

BEFORE AND AFTER

WE ARE ESSENTIAL TO ALBERTA | NAIT

Before lifting

- Child owns state
- Sibling components cannot access it

After lifting

- Parent owns state
- Children receive state via props
- Children notify parent via callbacks

REACT DATA FLOW

One-way data flow

- Data flows down via props.
- Events flow up via callbacks.
- The parent coordinates updates.

EXAMPLE PATTERN

WE ARE ESSENTIAL TO ALBERTA | NAIT

Parent component:

```
const [value, setValue] = useState(initialValue);
```

Pass down:

```
<Child value={value} onChange={setValue} />
```

Child notifies parent:

```
onClick={() => onChange(newValue)}
```

CONDITIONAL RENDERING

Controlled means state owns the value

Render UI based on state.

Logical AND:

```
{condition && <Component />}
```

Ternary:

```
{condition ? <A /> : <B />}
```

UI should derive from state, not manual DOM changes.

DERIVED DATA

Avoid storing data that can be calculated.

Instead of storing filtered results in state:

```
const filtered = items.filter(...);
```

Derive it from existing state.

PROP DRILLING

When state is lifted high in the tree, props may need to pass through multiple layers.

This can:

- Increase complexity
- Reduce readability

But it preserves explicit data flow.

WHEN TO LIFT STATE

Lift state when:

- Multiple components depend on the same data
- One component's action affects another
- You see duplicated state

Do not lift state unnecessarily.

HANDS-ON PRACTICE

Exercise: Refactor your existing exercise project to lift state to the nearest common parent

Objective: Implement shared state across several components

- Obtain the exercise starter files
  1. Create state at the necessary level
  2. Pass state and setter functions to required child components
  3. Ask questions if you are unsure of what to do or how the code works.

COMMON MISTAKES

- Duplicating shared state
- Storing derived data in state
- Forgetting to pass callbacks
- Mutating state directly
- Lifting state too high without reason

QUICK POLL

WE ARE ESSENTIAL TO ALBERTA | NAIT

Where should shared state live?

A. In every component that uses it
B. In the nearest common parent
C. In a global variable
D. In local variables only

Lesson Summary

- In this lesson, we covered the following:
  1. Explain the concept of lifting state and when it should be applied
  2. Pass state and event handlers between parent and child components
  3. Identify problems caused by duplicated state
  4. Describe the challenges of prop drilling

- Complete lesson 13 checklist

EXIT TICKET

Submit:

- A short explanation of lifting state
- One code example showing parent to child data flow
- One potential drawback of prop drilling